

Object Detection for Self-Driving Cars

Semantic Image Segmentation with U-Net

A standalone teaching guide for beginners. It explains how a self-driving car understands a road scene pixel by pixel, and walks through the companion script `object_detection_self_driving.py`, which builds a working U-Net you can run yourself.

THREE THINGS TO REMEMBER

1) Semantic image segmentation predicts a label for EVERY SINGLE PIXEL in an image. 2) A U-Net uses an EQUAL number of convolutional (downsampling) blocks and transposed convolution (upsampling) blocks - that symmetry is its 'U' shape. 3) SKIP CONNECTIONS carry detail across the U to prevent loss of border-pixel information and to help reduce overfitting.

1. How Does a Car 'See' the Road?

A self-driving car relies on cameras, but a raw camera image is just a grid of colored dots. To drive safely the car must know what each part of the scene actually is: which dots are drivable road, which are other vehicles, which are pedestrians, lane markings, or sky.

There are different levels of 'seeing' an image. Knowing a photo simply *contains* a car is not enough; the car needs to know exactly *which pixels* are that car and where its edges are. That most detailed level is called semantic image segmentation.

SEMANTIC IMAGE SEGMENTATION

Semantic image segmentation predicts a class label for every single pixel in the image. The output is a 'mask' the same size as the input, where each pixel has been colored in according to what it belongs to - road, car, person, and so on.

Task	What it outputs	Detail level
Classification	One label for the whole image	Lowest
Object detection	Boxes around objects	Medium
Semantic segmentation	A label for every pixel	Highest

2. The U-Net: Shape Explains Everything

U-Net is the classic network for segmentation. It is named for its U shape, which comes from two mirror-image halves joined at the bottom:

- **The encoder** (the left side, going down): a stack of convolutional blocks that progressively SHRINK the image while learning *what* is in it. Shrinking lets deeper layers take in a wider view of the scene.
- **The decoder** (the right side, going up): a stack of transposed-convolution blocks that progressively GROW the features back to the original resolution, so the final mask has one prediction per original pixel.

EQUAL BLOCKS ON EACH SIDE

U-Net uses an EQUAL number of downsampling blocks and upsampling blocks. Every time the encoder halves the image, a matching decoder block later doubles it back. That one-to-one symmetry is what makes the architecture a 'U'.

3. Skip Connections: The Secret Ingredient

There is a problem with shrinking an image and growing it back: every time we pool the image smaller, we throw away fine spatial detail - exactly the information needed for sharp, correctly-placed object borders. If we only used the shrunk features, the predicted mask would come out blurry around the edges.

Skip connections solve this. Before each encoder block pools its output down, U-Net saves a copy. Later, the matching decoder block CONCATENATES that saved copy onto its upsampled features. The fine detail takes a shortcut straight across the U instead of being squeezed through the bottleneck.

WHY SKIP CONNECTIONS MATTER

Skip connections prevent the loss of border-pixel information that pooling would otherwise destroy, giving crisp object edges. They also act as a regularizer that helps reduce overfitting, because the decoder can lean on real preserved detail rather than memorizing.

In code, a skip connection is just a concatenate:

```
up = Conv2DTranspose(n_filters, 3, strides=(2,2),
                    padding='same')(expansive_input)

# the skip connection: encoder features joined to decoder features
merge = concatenate([up, contractive_input], axis=3)
```

4. Walking Through the Code

4.1 The encoder block (downsampling)

Two convolutions detect features, an optional dropout fights overfitting, and max pooling halves the size. The block returns the pooled output (to go deeper) AND a pre-pooling copy (the skip connection).

```
conv = Conv2D(n_filters, 3, activation='relu',
             padding='same',
             kernel_initializer='he_normal')(inputs)
conv = Conv2D(n_filters, 3, activation='relu',
             padding='same',
             kernel_initializer='he_normal')(conv)
if dropout_prob > 0:
    conv = Dropout(dropout_prob)(conv)
if max_pooling:
    next_layer = MaxPooling2D(pool_size=(2,2))(conv)
skip_connection = conv
```

4.2 The decoder block (upsampling)

Transposed convolution doubles the size, the skip connection is concatenated, and two convolutions refine the result.

4.3 Assembling the U

Five encoder blocks double the filters ($32 \rightarrow 512$); the last is the bottleneck with no pooling. Four decoder blocks halve the filters back down, each pulling in the matching encoder skip connection. A final 1×1 convolution outputs one score per class per pixel.

```
cblock4 = conv_block(cblock3[0], n_filters*8, dropout_prob=0.3)
cblock5 = conv_block(cblock4[0], n_filters*16,
                    dropout_prob=0.3, max_pooling=False)
ublock6 = upsampling_block(cblock5[0], cblock4[1], n_filters*8)
...
conv10 = Conv2D(n_classes, 1, padding='same')(conv9)
```

5. From Scores to a Mask

The network outputs, for every pixel, one score per class (23 classes in this example). To get the final answer we take the class with the highest score at each pixel using `argmax`. The result is the segmentation mask the car uses.

```
pred_mask = tf.argmax(pred_mask, axis=-1) # best class per pixel
pred_mask = pred_mask[..., tf.newaxis]
```

6. How to Run the Code

Step 1 - Install the requirements:

```
python -m pip install tensorflow numpy
```

Step 2 - Run the script:

```
python object_detection_self_driving.py
```

The script builds the U-Net and prints a model summary, then pushes one random test image through it. You should see output shaped (1, 96, 128, 23) - a 96x128 image with 23 class scores per pixel - reduced to a (96, 128, 1) mask of one label per pixel. No dataset is needed; the goal is to understand the architecture. To train it for real, you would compile with the Adam optimizer and a `SparseCategoricalCrossentropy` loss and call `model.fit()` on labeled driving images.

7. Reading the Output

When the script runs successfully it prints a full model summary followed by two shape lines. Here is how to make sense of what scrolls past.

7.1 The model summary table

Keras prints one row per layer with three useful columns: the layer's name and type, its output shape, and how many trainable parameters it holds. Reading the Output Shape column top to bottom tells the whole U-Net story. The spatial size shrinks on the way down, hits bottom at the bottleneck, then grows back up - while the channel (filter) count does the opposite:

Stage	Output shape (H x W x C)	What is happening
Input	96 x 128 x 3	The original RGB image.
Encoder 1	96 x 128 x 32	Features learned, then pooled to 48 x 64.
Encoder 2	48 x 64 x 64	Smaller, more filters.
Encoder 3	24 x 32 x 128	Smaller still.
Encoder 4	12 x 16 x 256	Dropout added here.
Bottleneck	6 x 8 x 512	Smallest size, most filters; the turn of the U.
Decoder 1	12 x 16 x 256	Upsampled + skip from encoder 4.
Decoder 2	24 x 32 x 128	Upsampled + skip from encoder 3.
Decoder 3	48 x 64 x 64	Upsampled + skip from encoder 2.
Decoder 4	96 x 128 x 32	Back to full size + skip from encoder 1.
Output	96 x 128 x 23	One score per class, per pixel.

SPOT THE SKIP CONNECTIONS IN THE PRINTOUT

Every 'Concatenate' row lists TWO entries in its 'Connected to' column - for example conv2d_transpose joined with dropout, and later transposes joined with conv2d_5, conv2d_3, and conv2d_1. Those second entries are the encoder outputs taking the shortcut across the U. Seeing two inputs feed one concatenate layer is the skip connection made visible.

7.2 The parameter count

At the bottom the summary reports about 8.64 million total parameters (roughly 33 MB), all trainable. These are the weights the network would adjust during training. A model this size is modest by modern standards but already capable of real segmentation work.

7.3 The two shape lines

After building the model, the script pushes one random test image through it and prints:

```
Raw output shape : (1, 96, 128, 23) (1, H, W, n_classes)
Predicted mask    : (96, 128, 1) (one class label per pixel)
```

The raw output gives 23 class scores for each of the 96 x 128 pixels. After taking the highest-scoring class at each pixel with `argmax`, that collapses to a single label per pixel - the (96, 128, 1) mask. That final mask is exactly what a self-driving car would color in to understand the road.

7.4 About the messages printed before the results

On many machines - especially Windows - TensorFlow prints several warnings and information lines before the summary. These look alarming but are almost always harmless. Here is what they mean.

WARNINGS YOU CAN SAFELY IGNORE

oneDNN custom operations are on... — TensorFlow is using an optimized math library. It only warns that results may differ by tiny floating-point amounts. To silence it, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0` before running.

This TensorFlow binary is optimized... / To enable the following instructions: AVX2 FMA... — Your CPU supports extra speed-up instructions that this prebuilt TensorFlow was not compiled to use. The model still runs correctly, just not at maximum possible speed. No action needed.

All log messages before absl::InitializeLog()... — A cosmetic note about when logging started. Safe to ignore.

TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11... — On Windows, recent TensorFlow runs on the CPU even if you have a GPU. For this small demo that is completely fine. To use a GPU you would run under WSL2 or install the TensorFlow-DirectML plugin.

None of these messages stop the script. As long as you see the model summary, the two shape lines, and the final 'Done.', everything worked.

8. Quick Glossary

Term	Meaning
Semantic segmentation	Predicting a class label for every pixel.
Mask	The output image of per-pixel labels.
Encoder / downsampling	Convolution + pooling that shrinks the image.
Decoder / upsampling	Transposed convolution that grows it back.
Skip connection	Encoder features passed directly to the decoder.
Bottleneck	The deepest block, where the U turns around.
Conv2DTranspose	A layer that increases spatial resolution.
Dropout	Randomly disabling neurons to reduce overfitting.

argmax	Picking the highest-scoring class per pixel.
--------	--